



Bitácora de experimentación

TP3 — Perceptrones · cómo fue evolucionando.

Sistemas de Inteligencia Artificial · Trabajo Práctico 3

ITBA · Primer Cuatrimestre 2026

Una crónica, en orden cronológico, de cada experimento que corrimos: qué probamos, qué esperábamos, qué obtuvimos, qué decidimos. Incluye los descubrimientos sorpresa, las hipótesis revisadas, y las decisiones que el camino fue forzando.

Companion técnico: `docs/informe_tp3.pdf`.

Datos crudos de los experimentos: `ejercicio2/output/` y `ejercicio3/output/` en el repo.

Índice

1. Punto de partida

Estado del repo al inicio de la experimentación:

- Engine `mlp/` ya construido y testeado (78 tests verde).
- Validación Ej 0 ya pasaba: AND con escalón (error 0) y XOR con MLP (configs `[2, 2, 1]` y `[2, 3, 2, 1]` convergiendo).
- Ej 1 (knowledge distillation) ya implementado por Tomás con scripts independientes.
- Cuatro configs de arquitectura para Ej 2 ya creados.

Lo que faltaba experimentar:

- Búsqueda completa de hiperparámetros para Ej 2 (Fase 1 + Fase 2).
- Final eval del Ej 2 contra `digits_test.csv`.
- Ej 3 con `more_digits.csv` y, eventualmente, Pack C.

APRENDIMOS

La estrategia que adoptamos fue **coarse-to-fine en dos fases**: una Fase 1 exploratoria con K-folds=1 (split simple) para barrer rápido, y una Fase 2 con K-fold=5 para validar las elecciones con rigor.

Esto reduce el espacio de búsqueda de $\mathcal{O}(n^4)$ (cuatro hiperparámetros independientes) a $\mathcal{O}(4n)$ (sweeps secuenciales).

2. Fase 1 — sweeps coarse-to-fine sobre Ej 2 (k_folds=1)

Cuatro sub-fases secuenciales: arquitectura → optimizador → learning rate → batch size. Cada sub-fase usa el ganador de la anterior. Split simple 80/20 estratificado para velocidad.

2.1 Sub-fase 1.1 — Arquitectura

Hipótesis inicial: no sabemos si el problema necesita una red profunda o si una capa oculta alcanza. Probamos cuatro variantes alrededor de `[784, 100, 10]`.

Config	val_acc	macro_f1	best_ep	tiempo
arch_128_64	0.9686	0.863	10	42.3 s
arch_100_50	0.9674	0.863	7	27.6 s
arch_100	0.9670	0.859	11	35.1 s
arch_50	0.9578	0.856	16	39.9 s

DECIDIMOS

Elegimos `arch_100_50` (`[784, 100, 50, 10]`). `arch_128_64` estaba 0.12 pp arriba, pero con std del orden de 0.4 % es **empate técnico**, y `arch_100_50` es 35 % más rápida y converge antes (`best_ep=7` vs 10).

2.2 Sub-fase 1.2 — Optimizador

Hipótesis inicial: basados en lo que dice el apunte y la literatura, esperamos que Adam gane y SGD vanilla se quede atrás. Pero queríamos ver el orden de magnitud.

Config	val_acc	macro_f1	best_ep	tiempo
adam (lr=0.001)	0.9674	0.863	7	29.0 s
momentum (lr=0.01, $\beta = 0.9$)	0.9634	0.860	18	45.9 s
sgd (lr=0.01)	0.9481	0.846	48	77.0 s

APRENDIMOS

SGD no convergió en 50 epochs. Llegó a accuracy 94.81 % con best_epoch=48 (todavía mejorando). Para que SGD vanilla compita necesitaríamos muchas más epochs o un schedule de LR.

Momentum quedó cerca pero con best_ep=18 (vs 7 de Adam). El learning rate adaptativo de Adam acelera la convergencia inicial.

2.3 Sub-fase 1.3 — Learning rate

Hipótesis inicial: el rango “típico” que sugiere el apunte es 10^{-4} a 10^{-2} . Probamos cinco valores cubriendo ese rango.

Config	val_acc	macro_f1	best_ep	tot_ep
lr = 0.001	0.9674	0.863	7	18
lr = 0.0005	0.9654	0.864	17	28
lr = 0.0001	0.9622	0.861	47	50
lr = 0.005	0.9602	0.857	4	15
lr = 0.01	0.9530	0.850	2	13

APRENDIMOS

La curva de val_acc en función del learning rate tiene la forma esperada: un **máximo claro en 10^{-3}** , con caída a la izquierda (lento) y a la derecha (overshooting).

$lr = 10^{-4}$ no converge en 50 epochs (best_ep=47, total_ep=50). $lr = 10^{-2}$ overshoot fuerte (best_ep=2). $lr = 5 \cdot 10^{-3}$ overshoot moderado.

2.4 Sub-fase 1.4 — Batch size

Hipótesis inicial: esperamos que batch chico converja antes (más actualizaciones por epoch) pero quizá con val_acc final similar.

Config	val_acc	macro_f1	best_ep	tiempo
batch = 16	0.9674	0.8646	3	12.3 s
batch = 64	0.9674	0.8635	7	31.0 s
batch = 32	0.9658	0.8640	7	94.2 s
batch = 128	0.9658	0.8628	11	21.4 s

DECIDIMOS

batch=16 con **best_epoch=3**, mejor macro_f1 (0.8646), y el más rápido por bastante (12.3 s). El gradiente más ruidoso lleva a converger antes.

2.5 Consolidación: base.json con K-fold=5

Con los cuatro ganadores, armamos `base.json`: arquitectura [784, 100, 50, 10], Adam con $lr = 10^{-3}$, batch=16, 50 epochs con early stopping (patience=10). Lo corrimos con K-fold=5 antes de Fase 2.

	mean	std
val_acc_final	0.9622	0.0043
macro_f1	0.857	0.006
best_epoch	4.6	1.0

APRENDIMOS

El baseline con K-fold=5 confirmó la elección: **96.22 % $\pm$ 0.43 %**, std bajo. Convergencia muy rápida (best_ep=4.6 promedio). Este es el modelo que defendemos.

3. Fase 2 — one-at-a-time con K-fold=5

Sobre el `base.json` consolidado, variamos un solo HP a la vez para tener comparativas robustas. Quince corridas en total.

3.1 LR sweep (5 valores con extremos)

Config	val_acc	macro_f1	best_ep	tiempo (mean)
lr = 0.001	0.9622	0.857	4.6	449 s
lr = 0.0001	0.9579	0.852	31.6	1390 s
lr = 0.01	0.9471	0.839	4.6	455 s
lr = 0.1	0.2900	0.158	7.2	444 s
lr = 10	0.1248	0.028	11.6	689 s

SORPRESA

$lr = 10$ deja la red en **accuracy random (12.5 %)**. El loss explota a NaN tras pocas iteraciones; las predicciones quedan colapsadas.

$lr = 0,1$ no es tan catastrófico pero diverge claramente (29 % accuracy).

3.2 Arquitectura sweep (4 variantes)

Config	val_acc	macro_f1	best_ep	tiempo (mean)
arch_wider [200, 100]	0.9639	0.860	4.0	3445 s
arch_base [100, 50]	0.9622	0.857	4.6	945 s
arch_deeper [100, 50, 25]	0.9620	0.856	4.0	876 s
arch_shallow [30]	0.9515	0.846	7.6	17 s

APRENDIMOS

`arch_wider` gana por 0.17 pp **pero tarda** 3,6× **más** en entrenar (3445 s vs 945 s). Empate técnico con la base $\Rightarrow$ parsimonia.

`arch_shallow` corre en 17 segundos pero pierde 1 pp: claramente subajustado.

3.3 Optimizador sweep (3)

Config	val_acc	macro_f1	best_ep	tiempo
momentum	0.9629	0.857	6.0	292 s
adam	0.9622	0.857	4.6	417 s
sgd	0.9586	0.853	36.4	691 s

3.4 Inicializador sweep (3)

Config	val_acc	macro_f1	std (val_acc)
he	0.9602	0.854	0.003
xavier	0.9598	0.854	0.004
uniform	0.9594	0.854	0.002

APRENDIMOS

Empate total entre los tres inicializadores, std del orden de 0.3 %. La elección no es un cuello de botella en este problema.

La conclusión transversal de Fase 2 es que **Fase 2 valida más que descubre**. Solo el sweep de LR muestra rangos catastróficos; los demás quedan en empate técnico. La metodología coarse-to-fine de Fase 1 ya hizo el grueso del trabajo.

4. Final eval Ej 2 — el cachetazo

Con `base.json` congelado, corrimos el final eval: entrenar con todo `digits.csv` y evaluar UNA SOLA VEZ contra `digits_test.csv`.

val K-fold=5	0.9622
test (<code>digits_test.csv</code>)	0.8630
Drop	-10 pp

SORPRESA

Drop de 10 puntos porcentuales entre val K-fold=5 y test. Diez veces más grande que el std del K-fold (0.43 %). Imposible que sea ruido.

La matriz de confusión sobre test confirma que las confusiones se reparten por todo el grid: no es una clase específica que falla, es **patrón de distribution shift**.

APRENDIMOS

La interpretación: **el modelo está bien, los datos no se parecen**. `digits.csv` y `digits_test.csv` no son muestras de la misma distribución. Si esto es cierto, la solución no es un modelo más grande sino **más datos que cubran la distribución del test**.

Por suerte, eso es justo lo que el Ej 3 nos provee.

5. Ej 3 — la búsqueda del 98 %

5.1 Movimiento 1 — más datos, sin tocar el modelo

Hipótesis: si el problema es shift, agregar `more_digits.csv` (15.742 muestras adicionales) debería levantar el test sin necesidad de tocar la regularización.

Cambio único: agregar `.extra_csv_paths`: `["data and documentation/more_digits.csv"]` al config. Cero cambios al modelo.

	val K=5	test
Ej 2 base	0.9622	0.8630
Ej 3 base_extra_data	0.9706	0.9636
Δ	+0,84 pp	+10,06 pp

SORPRESA

+10 puntos porcentuales en test, sin tocar el modelo.

Macro F1 saltó de 0.857 a 0.959. Inspeccionando: la clase 8, que en Ej 2 quedaba con `precision = recall = 0` (la red nunca la predecía), **ahora se predice correctamente**. Más datos cubrieron el agujero de cobertura.

APRENDIMOS

La hipótesis del shift se confirmó. Pero no llegamos al target del 98 %: faltan 1.64 pp en val K-fold (97.06 % vs 98 %) y, sobre todo, 1.64 pp en test (96.36 % vs 98 %).

5.2 Movimiento 2 — Pack C: probar regularización

Sobre `base_extra_data`, activamos las técnicas Pack C **una por una y en combinaciones**.

5.2.1 L2 (weight decay $1 \cdot 10^{-4}$)

	val K=5	test
base_extra_data	0.9706	0.9636
+ L2 (1e-4)	0.9758	0.9656
Δ	+0,52 pp	+0,20 pp

L2 mejoró tanto val como test, aunque la mejora en test es modesta. Faltan 1.44 pp para el target.

5.2.2 Dropout ($p = 0,2$)

Hipótesis: dropout es la otra técnica clásica de regularización. Esperamos un efecto similar a L2.

	val K=5	test
base_extra_data	0.9706	0.9636
+ dropout (p=0.2)	0.9750	0.9628
Δ	+0,44 pp	-0,08 pp

SORPRESA

Dropout gana 0.44 pp en val pero pierde 0.08 pp en test.

La regularización ayudó a no sobreajustar a un fold particular, pero *no transfirió* al test.

Esto es coherente con la hipótesis del shift: dropout reduce varianza *dentro* de la distribución del train, pero el test viene de una distribución parcialmente distinta.

5.2.3 L2 + dropout (combinado)

Hipótesis: si L2 ayuda y dropout también, combinarlos debería mejorar.

	val K=5	test
base_extra_data	0.9706	0.9636
+ L2 + dropout	0.9772	0.9604
Δ	+0,66 pp	-0,32 pp

SORPRESA

Aún peor en test (-0,32 pp). Combinar L2 y dropout sobre-regulariza: gana en val (donde la regularización ayuda) pero pierde en test (donde el problema es otro).

5.2.4 Augmentation gaussiana ($\sigma = 0,05$, sobre L2)

Hipótesis: si el problema es shift, simular variabilidad en el input durante training podría ayudar.

Nota técnica: este sweep requirió **implementar augmentation gaussiana en el engine**, que no estaba en Phase A. Implementamos también dropout (con cache extendido para que el backward maneje el mask) y lr_schedule durante esta fase.

	val K=5	test
+ L2	0.9758	0.9656
+ L2 + aug ($\sigma = 0,05$)	0.9760	0.9688
Δ vs L2 solo	+0,02 pp	+0,32 pp

APRENDIMOS

La augmentation aporta sobre L2 en test (+0.32 pp adicional sobre L2 solo).

Sinergia: aug solo no funcionaría tanto, pero combinada con L2 sí.

Llegamos a 96.88 % en test. Faltan 1.12 pp para el 98 %.

5.2.5 Augmentation $\sigma = 0,10$ — ¿más ruido es mejor?

Hipótesis: si $\sigma = 0,05$ aporta, $\sigma = 0,10$ podría aportar más.

	val K=5	test
+ L2 + aug ($\sigma = 0,05$)	0.9760	0.9688
+ L2 + aug ($\sigma = 0,10$)	0.9768	0.9672
Δ	+0,08 pp	-0,16 pp

Más ruido no es mejor. $\sigma = 0,10$ degrada test apenas, $\sigma = 0,05$ se mantiene como ganador.

5.2.6 L2 + dropout + augmentation — todo combinado

	val K=5	test
+ L2 + aug	0.9760	0.9688
+ L2 + aug + dropout	0.9768	0.9656

Agregar dropout sobre L2+aug **empeora test** (−0,32 pp). Confirma que dropout en este problema no transfiere.

5.2.7 Wider + L2 + aug — más capacidad

Hipótesis: si la regularización limita y necesitamos más expresividad, una arquitectura más grande con regularización podría romper el techo.

	val K=5	test
+ L2 + aug ([100, 50])	0.9760	0.9688
+ wider [200, 100] + L2 + aug	0.9777	0.9640
Δ vs L2+aug	+0,17 pp	−0,48 pp

SORPRESA

Más capacidad mejora en val pero empeora en test.

Esto cierra el caso: el problema no es capacidad, es generalización. Más parámetros memorizan el train pero no transfieren al test que viene de distribución parcialmente distinta.

5.3 Resumen final del Ej 3

Config	val K=5	test	Δ vs base_extra
base_extra_data	0.9706	0.9636	— (referencia)
+ L2 (1e-4)	0.9758	0.9656	+0,20 pp
+ dropout (p=0.2)	0.9750	0.9628	−0,08 pp
+ L2 + dropout	0.9772	0.9604	−0,32 pp
+ L2 + aug ($\sigma = 0,05$)	0.9760	0.9688	+0,52 pp
+ L2 + aug ($\sigma = 0,10$)	0.9768	0.9672	+0,36 pp
+ L2 + aug + dropout	0.9768	0.9656	+0,20 pp
+ wider + L2 + aug	0.9777	0.9640	+0,04 pp

Ganador: 12_aug — L2 (1e-4) + augmentation gaussiana ($\sigma = 0,05$). Test **0.9688**.

Target del 98 % no alcanzado. Mejor: 96.88 %, faltan 1.12 pp.

6. Incidentes técnicos durante la experimentación

No todo fue limpio. Tres cosas valen la pena anotar para próximas iteraciones:

6.1 Multiprocessing fallando silenciosamente

A partir del cuarto o quinto sweep largo de Fase 2, los jobs lanzados con `-workers=5` empezaron a terminar con `exit 0` **sin escribir todos los CSVs**. El proceso multiprocessing iniciaba los workers, pero alguno moría sin reportar error visible.

Workaround: bajar a `-workers=1` (folds en serie). Más lento (5x), pero confiable. Las corridas afectadas se re-ejecutaron en serie y completaron correctamente.

6.2 final_eval.py con ModuleNotFoundError

La primera vez que corrimos `python3 ejercicio2/final_eval.py` se rompió con `ModuleNotFoundError: No module named 'mlp'`, porque el script vive en una subcarpeta y Python no incluye la raíz del proyecto en `sys.path`.

Fix: agregar `sys.path.insert(0, str(Path(__file__).resolve().parent.parent))` al inicio de `final_eval.py`. Aplicado a Ej 2 y Ej 3. Tests siguen verde (78/78).

6.3 Implementación de Pack C en el engine

L2 ya estaba en Phase A. Pero **dropout**, **augmentation gaussiana** y **lr_schedule** no estaban implementados. Los agregamos durante la iteración Pack C:

- **Dropout:** extendimos el cache del forward a `(z, a_pre, mask)` para que el backward use la mask en chain rule. Tests existentes seguían verde después de actualizar `test_forward_cache_pre_and_post_` a la nueva 3-tupla.
- **Augmentation gaussiana:** agregada al inicio del loop de mini-batches, antes del forward.
- **LR schedule (step decay):** agregado al final de cada época, antes del check de early stopping.

7. Aprendizajes generales

APRENDIMOS

1. Una buena pipeline gana más que tunear hiperparámetros.

El +10 pp del Ej 3 vino de *datos* (`more_digits.csv`), no de modelo. La regularización aportó solo +0,5 pp adicional. La lección: cuando hay un drop train→test grande, mirar primero la distribución de los datos, no la arquitectura.

APRENDIMOS

2. Los empates técnicos son más comunes que los wins claros.

La mayoría de los sweeps que corrimos tienen ganadores con margen < 0,3 pp y std del orden de 0,4 %. En esos casos, hay que elegir por **parsimonia** (más simple, más rápido), no por 0,05 pp en val. Si no, estás haciendo cherry-picking de ruido.

APRENDIMOS

3. “Mejor en val” no es “mejor en test” cuando hay shift.

Dropout fue el ejemplo de manual: ganador consistente en val K-fold (+0,44 pp) y *perdedor* en test (−0,08 pp). La razón: dropout reduce la varianza dentro de la distribución del train, pero no compensa el shift train→test. Con K-fold (que comparte distribución), funciona; con `final_eval` (que la cambia), no.

APRENDIMOS

4. Iterar técnicas de a una, no todas juntas.

`L2+dropout+aug` fue *peor* que `L2+aug` solas. Combinar regularizaciones cuando ya hay suficiente puede degradar. Si tuviéramos que repetir, mantendríamos el principio: agregar de a una y medir el delta antes de combinar.

APRENDIMOS

5. La hipótesis del techo del 98 % es coherente con los datos.

`wider` mejorando val pero empeorando test descarta que el problema sea capacidad. Augmentation gaussiana (isotrópica) ayuda apenas; el shift parece ser *geométrico* (estilo de escritura: rotación, traslación, grosor de trazo). El próximo paso natural sería augmentation por transformaciones geométricas — no implementada en este TP.

Fin de la bitácora. Datos crudos de cada experimento (CSVs por fold, epoch history, weights .npz) en `ejercicio2/output/` y `ejercicio3/output/` del repo.